

CHAPTER 9

Shading (Flat, Gouraud, Phong)

Expected Final Competency (KAD):

Students will be able to apply shading techniques to create lighting effects on 3D objects.

Materials:

1. Shading
2. Flat Shading
3. Gouraud Shading
4. Phong Shading
5. A Side-by-Side Comparison Between Flat, Gouraud, and Phong Shading
6. Implementasi shading (flat, gouraud, dan phong) pada objek 3D.
7. Soal Latihan

9.1 Shading

In computer graphics, shading refers to the techniques used to determine the color and brightness of surfaces in a 3D scene. The process involves simulating the interaction between light and objects to create realistic or stylistic visual effects.

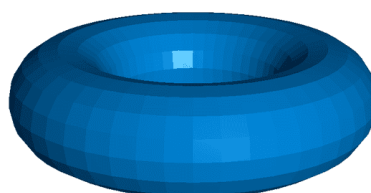
Shading is crucial for adding depth, detail, and texture to 3D models, making them appear more lifelike. Various shading techniques have been developed over the years, each with its own characteristics and computational requirements. Among the most common shading techniques are Flat shading, Gouraud shading, and Phong shading.

In this tutorial, we'll provide a comprehensive introduction and comparison between the three shading techniques.

9.2 Flat Shading

Flat shading is one of the simplest and earliest techniques used in computer graphics to render 3D models. Developed during the early days of computer graphics, flat shading calculates the color of each polygon face based on a single lighting calculation, applying this color uniformly across the entire face. This method provides a straightforward way to depict 3D shapes without the computational complexity associated with more advanced shading techniques.

In flat shading, the color of each polygon is determined by the angle between the polygon's surface normal and the light source. The surface normal is a vector perpendicular to the polygon's face, and it plays a crucial role in determining how light interacts with the surface. The lighting calculation typically involves the dot product of the surface normal and the light direction vector, which determines the intensity of the light hitting the surface. This intensity value is then used to shade the entire polygon uniformly:



Characteristics of flat shading:

- **Calculation:** One lighting calculation per polygon
- **Appearance:** Produces a faceted look with sharp edges between polygons, as each face has a uniform color
- **Performance:** Fastest among the three techniques, as it involves the least number of calculations
- **Use case:** Suitable for simple, low-polygon models where a faceted look is acceptable or desired

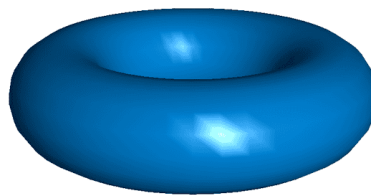
Flat shading is beneficial in situations where computational efficiency is more important than visual realism. It can be used in applications such as wireframe models, early video games, or any scenario where speed is a priority over detailed shading.

The below simple Python snippet simulates flat shading on a 3D object using the `flat_shading` function. In flat shading, each polygonal face of the object is shaded uniformly, based on the angle between the face's normal vector and a `light_source` direction:

```
def flat_shading(vertices, faces, light_source):
    shaded_faces = []
    for face in faces:
        normal = calculate_normal(face, vertices)
        intensity = np.dot(normal, light_source)
        color = np.clip(intensity, 0, 1)
        shaded_faces.append((face, color))
    return shaded_faces
```

9.3 Gouraud Shading

Gouraud shading, named after Henri Gouraud, a French computer scientist who introduced it, improves upon flat shading by providing smoother transitions between colors. In Gouraud shading, the lighting calculations are performed at the vertices of the polygons. The colors at the vertices are then interpolated across the surface of the polygons, creating a gradient effect, as illustrated below:



Characteristics of Gouraud Shading:

- **Calculation:** Lighting calculations are performed at the vertices, and colors are interpolated across the faces
- **Appearance:** Produces smoother transitions between colors across the surface, reducing the faceted look seen in flat shading
- **Performance:** Slower than flat shading but faster than Phong shading, as it involves interpolation but fewer lighting calculations
- **Use case:** Suitable for models where smooth shading is desired without the computational expense of Phong shading

Gouraud shading is widely used in real-time rendering applications such as video games and interactive graphics, where a balance between visual quality and computational efficiency is necessary.

The `gouraud_shading` function below simulates Gouraud shading on a 3D object. Gouraud shading interpolates the shading across the vertices of the faces:

```
def gouraud_shading(vertices, faces, light_source):
    vertex_colors = np.zeros(len(vertices))
    vertex_count = np.zeros(len(vertices))
    for face in faces:
        normal = calculate_normal(face, vertices)
        intensity = np.dot(normal, light_source)
        for vertex in face:
            vertex_colors[vertex] += intensity
            vertex_count[vertex] += 1

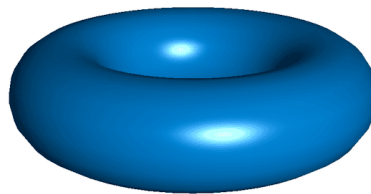
    vertex_colors /= vertex_count

    shaded_faces = []
    for face in faces:
        face_colors = [vertex_colors[vertex] for vertex in face]
        color = np.mean(face_colors)
        shaded_faces.append((face, color))

    return shaded_faces
```

9.4 Phong Shading

Phong shading, named after Bui Tuong Phong, a Vietnamese-born computer graphics researcher who developed it, provides the most realistic and detailed shading among the three techniques (see the illustration below). Unlike flat and Gouraud shading, Phong shading performs lighting calculations per pixel. It interpolates the surface normals across the polygon and calculates the color at each pixel using these interpolated normals:



Characteristics of Phong Shading:

- **Calculation:** Lighting calculations are performed per pixel, using the interpolated normals
- **Appearance:** Produces the most realistic and smooth shading, with highlights and reflections accurately rendered
- **Performance:** Slowest among the three techniques, as it involves the most calculations
- **Use case:** Ideal for high-quality renderings where realism is important, such as in animations, games, and detailed models

Phong shading is often used in scenarios where high visual fidelity is crucial, such as in animated films, high-end video games, and detailed scientific visualizations.

The below `phong_shading` function simulates Phong shading on a 3D object. Phong shading provides smooth shading across the surface of the object by interpolating normal vectors and computing the light intensity per vertex:

```
def phong_shading(vertices, faces, light_source):
    vertex_normals = np.zeros(vertices.shape)
    for face in faces:
        normal = calculate_normal(face, vertices)
        for vertex in face:
            vertex_normals[vertex] += normal

    vertex_normals /= np.linalg.norm(vertex_normals, axis=1)[:, np.newaxis]

    shaded_faces = []
    for face in faces:
        face_colors = [np.dot(vertex_normals[vertex], light_source) for
        vertex in face]
        color = np.mean(face_colors)
        shaded_faces.append((face, color))

    return shaded_faces
```

9.5 A Side-by-Side Comparison Between Flat, Gouraud, and Phong Shading

Here is a detailed side-by-side comparison between Flat, Gouraud, and Phong shading techniques:

Feature/Aspect	Flat Shading	Gouraud Shading	Phong Shading
Calculation point	Per polygon	Per vertex	Per pixel
Method	Single lighting calculation for the entire polygon	Lighting calculations at vertices, interpolated across the polygon	Interpolated normals, lighting calculated at each pixel
Color calculation	Uniform color across the polygon	Interpolated color from vertices across the polygon	Color calculated at each pixel for detailed shading
Appearance	Faceted look with sharp edges between polygons	Smooth transitions between colors, reduces faceted look	Smooth, realistic shading with accurate highlights and reflections
Performance	Fastest due to minimal calculations	Moderate speed, more calculations than Flat but fewer than Phong	Slowest due to per-pixel calculations and normal interpolation
Visual Quality	Low, with visible polygon edges	Moderate, with smoother color transitions	High, with realistic rendering and smooth gradients
Use Cases	Suitable for low-polygon models, simple graphics	Real-time rendering, video games, moderate detail models	High-quality renderings, animations, detailed models

Advantages	– Very fast	– Good balance between quality and performance	– High realism
	– Low computational cost	– Smoother appearance than Flat	– Accurate highlights and reflections
Disadvantages	– Faceted appearance	– Still not perfectly smooth	– High computational cost
	– Low realism	– Not as detailed as Phong	– Slower performance
Applications	Early video games, wireframe models, quick previews	Modern video games, interactive applications, real-time graphics	Animated films, high-end video games, scientific visualizations

9.6 Conclusion

In this chapter, we provided a brief introduction to three well-known shading techniques in computer graphics: Flat, Gouraud, and Phong. Each technique each offers different trade-offs between computational efficiency and visual quality, and each plays a pivotal role in determining the visual quality and realism of the 3D scene.

Flat shading is the simplest and fastest, suitable for applications where speed is critical. Gouraud shading offers a good balance between smooth appearance and performance, making it ideal for real-time applications. Phong shading, though computationally intensive, provides the highest level of realism, making it the preferred choice for high-quality renderings. Understanding these shading techniques and their applications helps in choosing the right approach for different graphical needs.

9.7 Flat Shading Sphere

Berikut contoh kode lengkap Java (Processing) untuk menampilkan objek sphere dengan efek *Flat Shading* di Processing IDE (mode P3D). Kode ini menghitung normal permukaan setiap segitiga (*face*) dari bola dan memberi warna berdasarkan hasil pencahayaan sekali per-polygon (*flat shading*). Kode akan menampilkan sphere 3D dengan efek *flat shading* (setiap segitiga memiliki warna pencahayaan seragam, sehingga permukaan tampak seperti bidang-bidang datar).

1. Sphere dibangun dari grid longitude × latitude.
2. Tiap segitiga dihitung normal bidangnya.
3. Warna tiap segitiga ditentukan oleh model pencahayaan sederhana (*ambient + diffuse*).
4. Warna tersebut diaplikasikan ke seluruh segitiga secara seragam (*flat shading*).

```
// =====
// Flat Shading Sphere - Processing (Java mode)
// =====
// Menampilkan bola 3D dengan pencahayaan flat shading
// =====

int res = 30; // resolusi grid bola (semakin besar semakin halus)
float radius = 150;

PVector lightDir;

void setup() {
  size(700, 700, P3D);
  noStroke();
  lightDir = new PVector(0.5, 0.3, 1).normalize(); // arah cahaya
```

```

}

void draw() {
    background(30);
    translate(width/2, height/2, 0);
    rotateY(frameCount * 0.01);
    rotateX(PI/6);

    drawFlatShadedSphere(radius, res);
}

void drawFlatShadedSphere(float r, int res) {
    float dTheta = TWO_PI / res;
    float dPhi = PI / res;

    // iterasi setiap segmen sphere
    for (int i = 0; i < res; i++) {
        float theta1 = i * dTheta;
        float theta2 = (i + 1) * dTheta;

        for (int j = 0; j < res; j++) {
            float phi1 = j * dPhi;
            float phi2 = (j + 1) * dPhi;

            // hitung 4 titik (membentuk quad)
            PVector v1 = sphToCart(r, theta1, phi1);
            PVector v2 = sphToCart(r, theta2, phi1);
            PVector v3 = sphToCart(r, theta2, phi2);
            PVector v4 = sphToCart(r, theta1, phi2);

            // gambar 2 segitiga (v1,v2,v3) dan (v1,v3,v4)
            drawFlatTriangle(v1, v2, v3);
            drawFlatTriangle(v1, v3, v4);
        }
    }
}

PVector sphToCart(float r, float theta, float phi) {
    float x = r * cos(theta) * sin(phi);
    float y = r * cos(phi);
    float z = r * sin(theta) * sin(phi);
    return new PVector(x, y, z);
}

void drawFlatTriangle(PVector a, PVector b, PVector c) {
    // hitung normal bidang (flat normal)
    PVector n = PVector.sub(b, a).cross(PVector.sub(c, a)).normalize();

    // model pencahayaan sederhana
    float ambient = 0.2;
    float diffuse = max(0, n.dot(lightDir)) * 0.8;
    float intensity = constrain(ambient + diffuse, 0, 1);

    // warna dasar oranye
    int baseColor = color(255, 140, 60);
    int shadedColor = color(
        red(baseColor) * intensity,
        green(baseColor) * intensity,
        blue(baseColor) * intensity
    );

    fill(shadedColor);

    beginShape(TRIANGLES);
    vertex(a.x, a.y, a.z);
    vertex(b.x, b.y, b.z);
    vertex(c.x, c.y, c.z);
}

```

```
endShape();
}
```

- `sphToCart()` → mengubah koordinat bola (θ, ϕ) menjadi koordinat kartesian (x, y, z).
- `drawFlatTriangle()` → menghitung normal permukaan per segitiga, lalu menentukan warna berdasarkan pencahayaan **ambient + diffuse**.
- `drawFlatShadedSphere()` → menggambar bola sebagai kumpulan segitiga kecil.
- Setiap segitiga diberi warna seragam berdasarkan normal bidangnya → menghasilkan tampilan **flat shading**.
- Bola berputar pelan dengan tampilan seperti **permukaan facet (datarnya terlihat)**.
- Sumber cahaya dari arah (0.5, 0.3, 1).
- Tidak ada interpolasi warna antar-vertex (berbeda dari Gouraud atau Phong).

9.8 Gouraud Shading Sphere

Kode ini melanjutkan konsep dari *Flat Shading Sphere* sebelumnya, tetapi kali ini kita menghitung intensitas cahaya di tiap vertex (bukan per-segitiga), lalu Processing akan secara otomatis menginterpolasi warna antar-vertexs sehingga menghasilkan efek permukaan halus (*smooth shading*). Kode akan menampilkan *sphere 3D* dengan efek *Gouraud Shading*, transisi pencahayaan halus antar bidang, dengan pencahayaan dihitung per-vertex.

1. Hitung normal di setiap vertex (rata-rata dari normal semua segitiga yang menempel pada vertex).
2. Untuk setiap vertex, hitung intensitas cahaya berdasarkan model pencahayaan (*ambient + diffuse*).
3. Gambar segitiga dengan warna per-vertex → Processing akan interpolasi warna antar vertexs secara otomatis.

```
// =====
// Gouraud Shading Sphere - Processing (Java mode)
// =====
// Menampilkan bola 3D dengan efek Gouraud shading
// =====

int res = 30;           // resolusi grid bola
float radius = 150;     // jari-jari bola
PVector lightDir;       // arah cahaya

void setup() {
  size(700, 700, P3D);
  noStroke();
  lightDir = new PVector(0.5, 0.3, 1).normalize(); // arah datang cahaya
}

void draw() {
  background(30);
  translate(width/2, height/2, 0);
  rotateY(frameCount * 0.01);
  rotateX(PI/6);

  drawGouraudShadedSphere(radius, res);
}

void drawGouraudShadedSphere(float r, int res) {
  float dTheta = TWO_PI / res;
  float dPhi = PI / res;

  // buat grid vertexs dan simpan posisi + normal + intensitasnya
  PVector[][] vertices = new PVector[res+1][res+1];
```

```

PVector[][] normals = new PVector[res+1][res+1];
float[][] intensity = new float[res+1][res+1];

for (int i = 0; i <= res; i++) {
  float theta = i * dTheta;
  for (int j = 0; j <= res; j++) {
    float phi = j * dPhi;
    vertices[i][j] = sphToCart(r, theta, phi);
    normals[i][j] = vertices[i][j].copy().normalize(); // normal arah keluar
    intensity[i][j] = computeLight(normals[i][j]); // intensitas per-vertex
  }
}

// gambar permukaan bola (interpolasi warna per vertex)
for (int i = 0; i < res; i++) {
  for (int j = 0; j < res; j++) {
    PVector v1 = vertices[i][j];
    PVector v2 = vertices[i+1][j];
    PVector v3 = vertices[i+1][j+1];
    PVector v4 = vertices[i][j+1];

    float i1 = intensity[i][j];
    float i2 = intensity[i+1][j];
    float i3 = intensity[i+1][j+1];
    float i4 = intensity[i][j+1];

    // warna dasar
    int baseColor = color(255, 140, 60);

    // segitiga 1
    beginShape(TRIANGLES);
    fill(red(baseColor)*i1, green(baseColor)*i1, blue(baseColor)*i1);
    vertex(v1.x, v1.y, v1.z);
    fill(red(baseColor)*i2, green(baseColor)*i2, blue(baseColor)*i2);
    vertex(v2.x, v2.y, v2.z);
    fill(red(baseColor)*i3, green(baseColor)*i3, blue(baseColor)*i3);
    vertex(v3.x, v3.y, v3.z);
    endShape();

    // segitiga 2
    beginShape(TRIANGLES);
    fill(red(baseColor)*i1, green(baseColor)*i1, blue(baseColor)*i1);
    vertex(v1.x, v1.y, v1.z);
    fill(red(baseColor)*i3, green(baseColor)*i3, blue(baseColor)*i3);
    vertex(v3.x, v3.y, v3.z);
    fill(red(baseColor)*i4, green(baseColor)*i4, blue(baseColor)*i4);
    vertex(v4.x, v4.y, v4.z);
    endShape();
  }
}

PVector sphToCart(float r, float theta, float phi) {
  float x = r * cos(theta) * sin(phi);
  float y = r * cos(phi);
  float z = r * sin(theta) * sin(phi);
  return new PVector(x, y, z);
}

// model pencahayaan sederhana (ambient + diffuse)
float computeLight(PVector N) {
  float ambient = 0.2;
  float diffuse = max(0, N.dot(lightDir)) * 0.8;
  return constrain(ambient + diffuse, 0, 1);
}

```


Bagian	Fungsi
<code>sphToCart()</code>	Mengubah koordinat bola (θ , ϕ) menjadi koordinat 3D
<code>normals[i][j]</code>	Normal arah keluar pada tiap verteks
<code>computeLight()</code>	Menghitung intensitas cahaya per verteks (ambient + diffuse)
<code>fill(...) per vertex</code>	Processing otomatis menginterpolasi warna antar verteks (efek Gouraud)
<code>drawGouraudShadedSphere()</code>	Menggambarkan dua segitiga per quad pada permukaan bola

Hasilnya: bola tampak **halus**, tidak seperti facet pada flat shading — karena warna diinterpolasi antar verteks.

9.9 Phong Shading Sphere

Pada teknik *Phong Shading*, intensitas cahaya dihitung per pixel (fragment) dengan cara:

1. Interpolasi normal antar verteks (bukan warna seperti pada Gouraud).
2. Hitung pencahayaan lengkap (*ambient + diffuse + specular*) di setiap pixel berdasarkan normal hasil interpolasi.

Untuk mencapai ini di Processing, kita perlu menggunakan **PShader (GLSL)** — yaitu *custom vertex dan fragment shader*.

Kode akan menampilkan objek sphere 3D dengan efek *Phong shading*, menghasilkan permukaan yang halus dengan highlight (pantulan cahaya) yang realistis.

Buat tiga file di *folder sketch Processing* Anda:

```
PhongShadingExample/
├─ PhongShadingExample.pde
├─ phong.vert
└─ phong.frag
```

File 1 — PhongShadingExample.pde

```
// =====
// Phong Shading Sphere - Processing (Java mode)
// =====
// Menggunakan custom shader GLSL (phong.vert + phong.frag)
// =====

PShader phongShader;

void setup() {
  size(700, 700, P3D);
  noStroke();

  // load shader
  phongShader = loadShader("phong.frag", "phong.vert");

  // set parameter awal shader
  phongShader.set("lightPos", 500.0, 300.0, 600.0);
  phongShader.set("baseColor", 1.0, 0.55, 0.25); // oranye
  phongShader.set("ambientStrength", 0.2);
  phongShader.set("diffuseStrength", 0.8);
  phongShader.set("specularStrength", 0.5);
  phongShader.set("shininess", 32.0);
}

void draw() {
  background(25);
  shader(phongShader);
```

```
// atur posisi kamera dan view
translate(width/2, height/2, 0);
rotateY(frameCount * 0.01);
rotateX(PI/6);

// update posisi kamera ke shader (supaya highlight sesuai arah pandang)
phongShader.set("viewPos", 0.0, 0.0, 500.0);

// gambar sphere
sphereDetail(100);
sphere(150);

resetShader();
}
```

File 2 — phong.vert

Vertex Shader (GLSL) – menghitung posisi dan meneruskan normal & posisi verteks ke *fragment shader*.

```
#version 150

in vec4 position;
in vec3 normal;

uniform mat4 modelview;
uniform mat4 projection;
uniform mat3 normalMatrix;

out vec3 vNormal;
out vec3 vPosition;

void main() {
    vNormal = normalize(normalMatrix * normal);
    vPosition = vec3(modelview * position);
    gl_Position = projection * modelview * position;
}
```

File 3 — phong.frag

Fragment Shader (GLSL) – melakukan pencahayaan per-pixel (*Phong shading*).

```
#version 150

in vec3 vNormal;
in vec3 vPosition;

out vec4 fragColor;

uniform vec3 lightPos;
uniform vec3 viewPos;
uniform vec3 baseColor;

uniform float ambientStrength;
uniform float diffuseStrength;
uniform float specularStrength;
uniform float shininess;

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(lightPos - vPosition);
    vec3 V = normalize(viewPos - vPosition);
    vec3 R = reflect(-L, N);

    // komponen pencahayaan
    vec3 ambient = ambientStrength * baseColor;
```

```

float diff = max(dot(N, L), 0.0);
vec3 diffuse = diffuseStrength * diff * baseColor;

float spec = pow(max(dot(R, V), 0.0), shininess);
vec3 specular = specularStrength * spec * vec3(1.0);

vec3 result = ambient + diffuse + specular;
fragColor = vec4(result, 1.0);
}

```

Tahap	Deskripsi
Vertex Shader	Menghitung posisi vertex (<code>vPosition</code>) dan normal (<code>vNormal</code>).
Interpolasi Otomatis (GPU)	Normal diinterpolasi antar vertex oleh GPU untuk setiap pixel (fragment).
Fragment Shader	Di setiap pixel: hitung ambient + diffuse + specular berdasarkan normal hasil interpolasi.
Hasil Akhir	Highlight (pantulan cahaya) muncul halus, bahkan di tengah permukaan — karakteristik Phong shading.

Parameter yang Bisa Anda Eksperimenkan

Parameter	Keterangan
<code>lightPos</code>	Posisi sumber cahaya (x, y, z)
<code>viewPos</code>	Posisi kamera (biasanya di depan objek)
<code>ambientStrength</code>	Intensitas cahaya ambient
<code>diffuseStrength</code>	Intensitas cahaya diffuse
<code>specularStrength</code>	Intensitas pantulan (highlight)
<code>shininess</code>	Ukuran highlight (semakin besar semakin tajam)

9.10 Perbandingan Teknik Shading (*Flat*, *Gouraud*, *Phong*)

Berikut versi terpadu (*full version*) untuk menampilkan *Flat*, *Gouraud*, dan *Phong Shading* dalam satu jendela *Processing*.

Struktur Folder:

```

ShadingComparison/
├─ ShadingComparison.pde
├─ flat.vert
├─ flat.frag
├─ gouraud.vert
├─ gouraud.frag
├─ phong.vert
└─ phong.frag

```

File 1 — `ShadingComparison.pde`

```

// =====
// Perbandingan Flat, Gouraud, dan Phong Shading
// Processing (Java Mode)
// =====

PShader flatShader, gouraudShader, phongShader;

void setup() {
  size(1200, 600, P3D);
  noStroke();

  // Load semua shader

```

```

flatShader = loadShader("flat.frag", "flat.vert");
gouraudShader = loadShader("gouraud.frag", "gouraud.vert");
phongShader = loadShader("phong.frag", "phong.vert");

// Set parameter cahaya & material untuk semua shader
setShaderUniforms(flatShader);
setShaderUniforms(gouraudShader);
setShaderUniforms(phongShader);
}

void draw() {
    background(255);
    lights();
    float rot = frameCount * 0.01;

    // Posisi kamera pandang (untuk shading Phong)
    PVector viewPos = new PVector(0, 0, 500);
    phongShader.set("viewPos", viewPos.x, viewPos.y, viewPos.z);

    // ===== Flat Shading =====
    pushMatrix();
    translate(width * 0.25, height / 2, 0);
    rotateY(rot);
    shader(flatShader);
    sphereDetail(50);
    sphere(120);
    resetShader();
    fill(255);
    textAlign(CENTER);
    text("FLAT SHADING", width * 0.25, height - 20);
    popMatrix();

    // ===== Gouraud Shading =====
    pushMatrix();
    translate(width * 0.5, height / 2, 0);
    rotateY(rot);
    shader(gouraudShader);
    sphereDetail(50);
    sphere(120);
    resetShader();
    fill(255);
    text("GOURAUD SHADING", width * 0.5, height - 20);
    popMatrix();

    // ===== Phong Shading =====
    pushMatrix();
    translate(width * 0.75, height / 2, 0);
    rotateY(rot);
    shader(phongShader);
    sphereDetail(100);
    sphere(120);
    resetShader();
    fill(255);
    text("PHONG SHADING", width * 0.75, height - 20);
    popMatrix();
}

// Fungsi bantu untuk set parameter cahaya/material
void setShaderUniforms(PShader sh) {
    sh.set("lightPos", 500.0, 300.0, 600.0);
    sh.set("baseColor", 1.0, 0.55, 0.25); // oranye
    sh.set("ambientStrength", 0.2);
    sh.set("diffuseStrength", 0.8);
    sh.set("specularStrength", 0.5);
    sh.set("shininess", 32.0);
}

```

File 2 — flat.vert

```
#version 150
in vec4 position;
in vec3 normal;

uniform mat4 modelview;
uniform mat4 projection;
uniform mat3 normalMatrix;

out vec3 vNormal;
out vec3 vPosition;

void main() {
    vNormal = normalize(normalMatrix * normal);
    vPosition = vec3(modelview * position);
    gl_Position = projection * modelview * position;
}
```

File 3 — flat.frag

```
#version 150
in vec3 vNormal;
in vec3 vPosition;

out vec4 fragColor;

uniform vec3 lightPos;
uniform vec3 baseColor;
uniform float ambientStrength;
uniform float diffuseStrength;
uniform float specularStrength;
uniform float shininess;

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(lightPos - vPosition);
    vec3 R = reflect(-L, N);
    vec3 V = normalize(-vPosition);

    // Flat shading: hitung cahaya hanya satu kali (gunakan normal vertex)
    vec3 ambient = ambientStrength * baseColor;
    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diffuseStrength * diff * baseColor;
    float spec = pow(max(dot(R, V), 0.0), shininess);
    vec3 specular = specularStrength * spec * vec3(1.0);
    vec3 color = ambient + diffuse + specular;

    fragColor = vec4(color, 1.0);
}
```

File 4 — gouraud.vert

```
#version 150
in vec4 position;
in vec3 normal;

uniform mat4 modelview;
uniform mat4 projection;
uniform mat3 normalMatrix;

uniform vec3 lightPos;
uniform vec3 baseColor;
uniform float ambientStrength;
uniform float diffuseStrength;
uniform float specularStrength;
uniform float shininess;
```

```

out vec3 vertexColor;

void main() {
    vec3 N = normalize(normalMatrix * normal);
    vec3 fragPos = vec3(modelview * position);
    vec3 L = normalize(lightPos - fragPos);
    vec3 V = normalize(-fragPos);
    vec3 R = reflect(-L, N);

    vec3 ambient = ambientStrength * baseColor;
    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diffuseStrength * diff * baseColor;
    float spec = pow(max(dot(R, V), 0.0), shininess);
    vec3 specular = specularStrength * spec * vec3(1.0);

    vertexColor = ambient + diffuse + specular;
    gl_Position = projection * modelview * position;
}

```

File 5 — gouraud.frag

```

#version 150
in vec3 vertexColor;
out vec4 fragColor;

void main() {
    fragColor = vec4(vertexColor, 1.0);
}

```

File 6 — phong.vert

```

#version 150
in vec4 position;
in vec3 normal;

uniform mat4 modelview;
uniform mat4 projection;
uniform mat3 normalMatrix;

out vec3 vNormal;
out vec3 vPosition;

void main() {
    vNormal = normalize(normalMatrix * normal);
    vPosition = vec3(modelview * position);
    gl_Position = projection * modelview * position;
}

```

File 7 — phong.frag

```

#version 150
in vec3 vNormal;
in vec3 vPosition;
out vec4 fragColor;

uniform vec3 lightPos;
uniform vec3 viewPos;
uniform vec3 baseColor;
uniform float ambientStrength;
uniform float diffuseStrength;
uniform float specularStrength;
uniform float shininess;

```

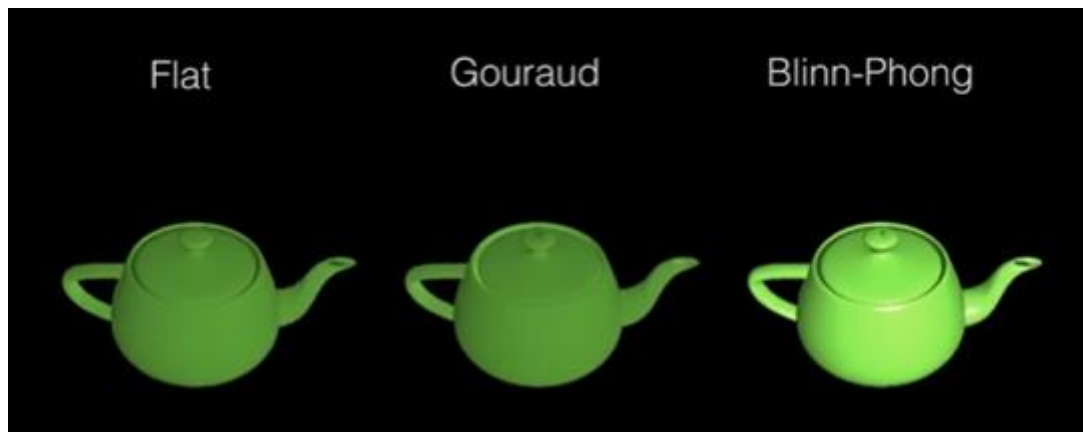
```
void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(lightPos - vPosition);
    vec3 V = normalize(viewPos - vPosition);
    vec3 R = reflect(-L, N);

    vec3 ambient = ambientStrength * baseColor;
    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diffuseStrength * diff * baseColor;
    float spec = pow(max(dot(R, V), 0.0), shininess);
    vec3 specular = specularStrength * spec * vec3(1.0);

    vec3 color = ambient + diffuse + specular;
    fragColor = vec4(color, 1.0);
}
```

9.11 Assignment

Buatlah objek teapot dengan tampilan *Flat*, *Gouraud*, dan *Phong Shading* seperti gambar berikut.



Source: <https://www.youtube.com/watch?v=VRw3GuVldo>

References:

1. Bui Tuong Phong, "Illumination for Computer Generated Pictures," Communications of the ACM, 1975.
2. Gambetta, Gabriel. 2021. *Computer graphics from scratch: a programmer's introduction to 3D rendering*. San Francisco: No Starch Press.
3. Gabriel Gambetta, *Computer Graphics from Scratch: Lighting and Shading* — gabrielgambetta.com
4. Gouraud & Phong Shading Notes — cglearn.eu
5. *Gouraud Shading in Computer Graphics*, geeksforgeeks.org
6. Gabriel Gambetta, *Computer Graphics from Scratch – Lighting and Shading* <https://gabrielgambetta.com/computer-graphics-from-scratch/13-shading.html>
7. CGLearn.eu – Shading and Lighting in Computer Graphics <https://cglearn.eu/pub/computer-graphics/shading-and-lighting>
8. *Shading and Lighting*, cglearn.eu
9. <https://www.baeldung.com/cs/shading-flat-vs-gouraud-vs-phong>
10. <https://processing.org/reference/>